

1. Criar a Interface Base

Define a estrutura comum para todas as bebidas.

```
public interface IBeverage
{
    string GetDescription();
    decimal GetCost();
}
```

2. Criar a Classe Base Concreta

Implementa a interface básica.

```
public class Coffee : IBeverage
{
    public string GetDescription()
    {
        return "Coffee";
    }

    public decimal GetCost()
    {
        return 5.00m; // Preço base do café
    }
}
```

3. Criar a Classe Base para os Decoradores

Todos os decoradores implementam a mesma interface e decoram uma bebida existente.

```
public abstract class BeverageDecorator : IBeverage
{
    protected IBeverage _beverage;

    protected BeverageDecorator(IBeverage beverage)
    {
        _beverage = beverage;
    }

    public virtual string GetDescription()
    {
        return _beverage.GetDescription();
    }

    public virtual decimal GetCost()
    {
        return _beverage.GetCost();
    }
}
```

4. Criar Decoradores Concretos

Cada decorador adiciona comportamento específico.

```
public class MilkDecorator : BeverageDecorator
{
    public MilkDecorator(IBeverage beverage) : base(beverage) { }

    public override string GetDescription()
    {
        return _beverage.GetDescription() + ", Milk";
    }

    public override decimal GetCost()
    {
        return _beverage.GetCost() + 1.50m; // Custo adicional do leite
    }
}

public class SugarDecorator : BeverageDecorator
{
    public SugarDecorator(IBeverage beverage) : base(beverage) { }

    public override string GetDescription()
    {
        return _beverage.GetDescription() + ", Sugar";
    }

    public override decimal GetCost()
    {
        return _beverage.GetCost() + 0.50m; // Custo adicional do açúcar
    }
}

public class SyrupDecorator : BeverageDecorator
{
    public SyrupDecorator(IBeverage beverage) : base(beverage) { }

    public override string GetDescription()
    {
        return _beverage.GetDescription() + ", Syrup";
    }

    public override decimal GetCost()
    {
        return _beverage.GetCost() + 2.00m; // Custo adicional da calda
    }
}
```

5. Usar o Decorator

```
class Program
{
    static void Main(string[] args)
    {
        // Criar um café simples
        IBeverage beverage = new Coffee();
        Console.WriteLine($"{beverage.GetDescription()} - {beverage.GetCost():C}");

        // Adicionar leite ao café
        beverage = new MilkDecorator(beverage);
        Console.WriteLine($"{beverage.GetDescription()} - {beverage.GetCost():C}");

        // Adicionar açúcar ao café com leite
        beverage = new SugarDecorator(beverage);
        Console.WriteLine($"{beverage.GetDescription()} - {beverage.GetCost():C}");

        // Adicionar calda ao café com leite e açúcar
        beverage = new SyrupDecorator(beverage);
        Console.WriteLine($"{beverage.GetDescription()} - {beverage.GetCost():C}");
    }
}
```

Saída Esperada

Coffee - \$5.00

Coffee, Milk - \$6.50

Coffee, Milk, Sugar - \$7.00

Coffee, Milk, Sugar, Syrup - \$9.00

Vantagens do Decorator Pattern

1. **Flexibilidade:** Permite combinar e empilhar comportamentos de forma dinâmica.
2. **Abstração:** O cliente não precisa saber como os objetos são decorados.
3. **Reuso:** Comportamentos podem ser reutilizados em diferentes combinações.

Quando evitar?

- Se o número de combinações de decoradores for muito alto, pode se tornar complexo e difícil de gerenciar.
- Se a hierarquia de classes for mais simples com herança direta.

Resumo

O **Decorator Pattern** permite adicionar funcionalidades a objetos sem alterar seu código ou criar subclasses. Ele é útil para cenários em que os requisitos mudam frequentemente ou há muitas combinações possíveis de comportamentos. □